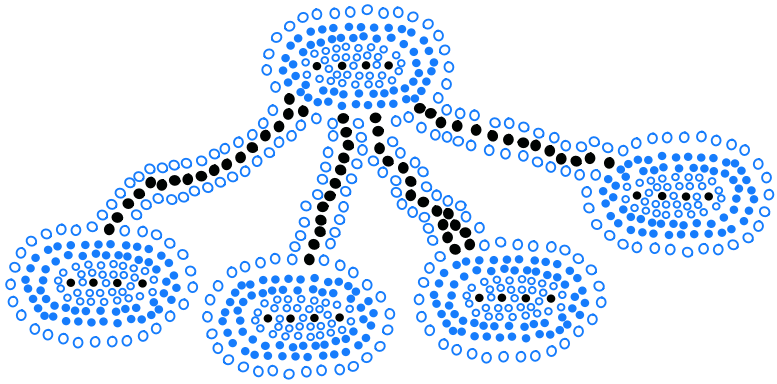
Chapter 15 Memory Management and B-Trees



Contents

15.1 Memory Management 688
15.1.1 Stacks in the Java Virtual Machine 688
15.1.2 Allocating Space in the Memory Heap 691
15.1.3 Garbage Collection 693
15.2 Memory Hierarchies and Caching 695
15.2.1 Memory Systems 695
15.2.2 Caching Strategies 696
15.3 External Searching and B-Trees 701
15.3.1 (a,b) Trees 702
15.3.2 B-Trees 704
15.4 External-Memory Sorting 705
15.4.1 Multiway Merging 706
15.5 Exercises 707

15.1 Memory Management

Computer memory is organized into a sequence of ***words***, each of which typically consists of 4, 8, or 16 bytes (depending on the computer). These memory words are numbered from 0 to $N-1$, where N is the number of memory words available to the computer. The number associated with each memory word is known as its ***memory address***. Thus, the memory in a computer can be viewed as basically one giant array of memory words, as portrayed in Figure 15.1.

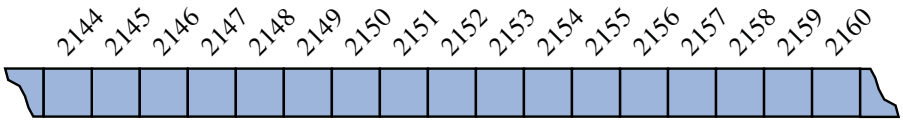


Figure 15.1: Memory addresses.

In order to run programs and store information, the computer's memory must be ***managed*** so as to determine what data is stored in what memory cells. In this section, we discuss the basics of memory management, most notably describing the way in which memory is allocated for various purposes in a Java program, and the way in which portions of memory are deallocated and reclaimed, when no longer needed.

15.1.1 Stacks in the Java Virtual Machine

A Java program is typically compiled into a sequence of byte codes that serve as "machine" instructions for a well-defined model—the ***Java Virtual Machine*** (***JVM***). The definition of the JVM is at the heart of the definition of the Java language itself. By compiling Java code into the JVM byte codes, rather than the machine language of a specific CPU, a Java program can be run on any computer that has a program that can emulate the JVM.

Stacks have an important application to the runtime environment of Java programs. A running Java program (more precisely, a running Java thread) has a private stack, called the ***Java method stack*** or just ***Java stack*** for short, which is used to keep track of local variables and other important information on methods as they are invoked during execution. (See Figure 15.2.)

More specifically, during the execution of a Java program, the Java Virtual Machine (JVM) maintains a stack whose elements are descriptors of the currently active (that is, nonterminated) invocations of methods. These descriptors are called ***frames***. A frame for some invocation of method "fool" stores the current values of the local variables and parameters of method fool, as well as information on method "cool" that called fool and on what needs to be returned to method "cool".

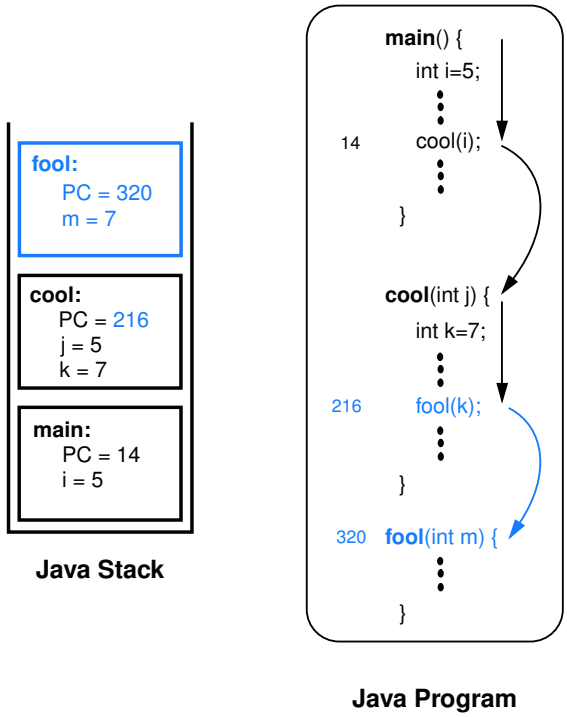


Figure 15.2: An example of a Java method stack: method fool has just been called by method cool, which itself was previously called by method main. Note the values of the program counter, parameters, and local variables stored in the stack frames. When the invocation of method fool terminates, the invocation of method cool will resume its execution at instruction 217, which is obtained by incrementing the value of the program counter stored in the stack frame.

Keeping Track of the Program Counter

The JVM keeps a special variable, called the ***program counter***, to maintain the address of the statement the JVM is currently executing in the program. When a method "cool" invokes another method "fool", the current value of the program counter is recorded in the frame of the current invocation of cool (so the JVM will know where to return to when method fool is done). At the top of the Java stack is the frame of the ***running method***, that is, the method that currently has control of the execution. The remaining elements of the stack are frames of the ***suspended methods***, that is, methods that have invoked another method and are currently waiting for it to return control to them upon its termination. The order of the elements in the stack corresponds to the chain of invocations of the currently active methods. When a new method is invoked, a frame for this method is pushed onto the stack. When it terminates, its frame is popped from the stack and the JVM resumes the processing of the previously suspended method.

Implementing Recursion

One of the benefits of using a stack to implement method invocation is that it allows programs to use ***recursion***. That is, it allows a method to call itself, as discussed in Chapter 5. We implicitly described the concept of the call stack and the use of frames within our portrayal of ***recursion traces*** in that chapter. Interestingly, early programming languages, such as Cobol and Fortran, did not originally use call stacks to implement function and procedure calls. But because of the elegance and efficiency that recursion allows, all modern programming languages, including the modern versions of classic languages like Cobol and Fortran, utilize a runtime stack for method and procedure calls.

Each box of a recursion trace corresponds to a frame of the Java method stack. At any point in time, the contents of the Java method stack corresponds to the chain of boxes from the initial method invocation to the current one.

To better illustrate how a runtime stack allows recursive methods, we refer back to the Java implementation of the classic recursive definition of the factorial function,

$$n! = n(n-1)(n-2)\cdots 1,$$

with the code originally given in Code Fragment 5.1, and the recursion trace in Figure 5.1. The first time we call method factorial(n), its stack frame includes a local variable storing the value n. The method recursively calls itself to compute $(n-1)!$, which pushes a new frame on the Java runtime stack. In turn, this recursive invocation calls itself to compute $(n-2)!$, etc. The chain of recursive invocations, and thus the runtime stack, only grows up to size $n+1$, with the most deeply nested call being factorial(0), which returns 1 without any further recursion. The runtime stack allows several invocations of the factorial method to exist simultaneously. Each has a frame that stores the value of its parameter n as well as the value to be returned. When the first recursive call eventually terminates, it returns $(n-1)!$, which is then multiplied by n to compute $n!$ for the original call of the factorial method.

The Operand Stack

Interestingly, there is actually another place where the JVM uses a stack. Arithmetic expressions, such as $((a+b)*(c+d))/e$, are evaluated by the JVM using an ***operand stack***. A simple binary operation, such as $a+b$, is computed by pushing a on the stack, pushing b on the stack, and then calling an instruction that pops the top two items from the stack, performs the binary operation on them, and pushes the result back onto the stack. Likewise, instructions for writing and reading elements to and from memory involve the use of pop and push methods for the operand stack. Thus, the JVM uses a stack to evaluate arithmetic expressions in Java.

15.1.2 Allocating Space in the Memory Heap

We have already discussed (in Section 15.1.1) how the Java Virtual Machine allocates a method's local variables in that method's frame on the Java runtime stack. The Java stack is not the only kind of memory available for program data in Java, however.

Dynamic Memory Allocation

Memory for an object can also be allocated dynamically during a method's execution, by having that method utilize the special **new** operator built into Java. For example, the following Java statement creates an array of integers whose size is given by the value of variable k:

```
int[ ] items = new int[k];
```

The size of the array above is known only at runtime. Moreover, the array may continue to exist even after the method that created it terminates. Thus, the memory for this array cannot be allocated on the Java stack.

The Memory Heap

Instead of using the Java stack for this object's memory, Java uses memory from another area of storage—the ***memory heap*** (which should not be confused with the "heap" data structure presented in Chapter 9). We illustrate this memory area, together with the other memory areas, in a Java Virtual Machine in Figure 15.3. The storage available in the memory heap is divided into ***blocks***, which are contiguous array-like "chunks" of memory that may be of variable or fixed sizes.

To simplify the discussion, let us assume that blocks in the memory heap are of a fixed size, say, 1,024 bytes, and that one block is big enough for any object we might want to create. (Efficiently handling the more general case is actually an interesting research problem.)

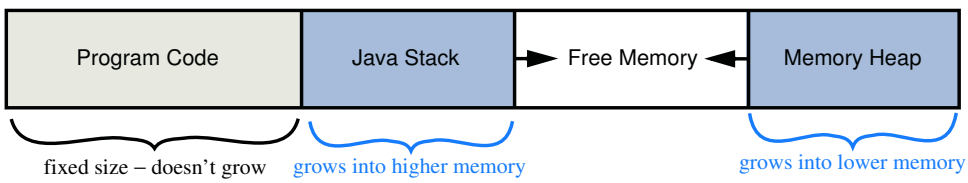


Figure 15.3: A schematic view of the layout of memory addresses in the Java Virtual Machine.

Memory Allocation Algorithms

The Java Virtual Machine definition requires that the memory heap be able to quickly allocate memory for new objects, but it does not specify the algorithm that should be used to do this. One popular method is to keep contiguous "holes" of available free memory in a linked list, called the ***free list***. The links joining these holes are stored inside the holes themselves, since their memory is not being used. As memory is allocated and deallocated, the collection of holes in the free lists changes, with the unused memory being separated into disjoint holes divided by blocks of used memory. This separation of unused memory into separate holes is known as ***fragmentation***. The problem is that it becomes more difficult to find large continuous chunks of memory, when needed, even though an equivalent amount of memory may be unused (yet fragmented).

Two kinds of fragmentation can occur. ***Internal fragmentation*** occurs when a portion of an allocated memory block is unused. For example, a program may request an array of size 1000, but only use the first 100 cells of this array. A runtime environment can not do much to reduce internal fragmentation. ***External fragmentation***, on the other hand, occurs when there is a significant amount of unused memory between several contiguous blocks of allocated memory. Since the runtime environment has control over where to allocate memory when it is requested (for example, when the **new** keyword is used in Java), the runtime environment should allocate memory in a way to try to reduce external fragmentation.

Several heuristics have been suggested for allocating memory from the heap so as to minimize external fragmentation. The ***best-fit algorithm*** searches the entire free list to find the hole whose size is closest to the amount of memory being requested. The ***first-fit algorithm*** searches from the beginning of the free list for the first hole that is large enough. The ***next-fit algorithm*** is similar, in that it also searches the free list for the first hole that is large enough, but it begins its search from where it left off previously, viewing the free list as a circularly linked list (Section 3.3). The ***worst-fit algorithm*** searches the free list to find the largest hole of available memory, which might be done faster than a search of the entire free list if this list were maintained as a priority queue (Chapter 9). In each algorithm, the requested amount of memory is subtracted from the chosen memory hole and the leftover part of that hole is returned to the free list.

Although it might sound good at first, the best-fit algorithm tends to produce the worst external fragmentation, since the leftover parts of the chosen holes tend to be small. The first-fit algorithm is fast, but it tends to produce a lot of external fragmentation at the front of the free list, which slows down future searches. The next-fit algorithm spreads fragmentation more evenly throughout the memory heap, thus keeping search times low. This spreading also makes it more difficult to allocate large blocks, however. The worst-fit algorithm attempts to avoid this problem by keeping contiguous sections of free memory as large as possible.

15.1.3 Garbage Collection

In some languages, like C and C++, the memory space for objects must be explicitly deallocated by the programmer, which is a duty often overlooked by beginning programmers and is the source of frustrating programming errors even for experienced programmers. The designers of Java instead placed the burden of memory management entirely on the runtime environment.

As mentioned above, memory for objects is allocated from the memory heap and the space for the instance variables of a running Java program are placed in its method stacks, one for each running thread (for the simple programs discussed in this book there is typically just one running thread). Since instance variables in a method stack can refer to objects in the memory heap, all the variables and objects in the method stacks of running threads are called ***root objects***. All those objects that can be reached by following object references that start from a root object are called ***live objects***. The live objects are the active objects currently being used by the running program; these objects should ***not*** be deallocated. For example, a running Java program may store, in a variable, a reference to a sequence S that is implemented using a doubly linked list. The reference variable to S is a root object, while the object for S is a live object, as are all the node objects that are referenced from this object and all the elements that are referenced from these node objects.

From time to time, the Java virtual machine (JVM) may notice that available space in the memory heap is becoming scarce. At such times, the JVM can elect to reclaim the space that is being used for objects that are no longer live, and return the reclaimed memory to the free list. This reclamation process is known as ***garbage collection***. There are several different algorithms for garbage collection, but one of the most used is the ***mark-sweep algorithm***.

The Mark-Sweep Algorithm

In the mark-sweep garbage collection algorithm, we associate a "mark" bit with each object that identifies whether that object is live. When we determine at some point that garbage collection is needed, we suspend all other activity and clear the mark bits of all the objects currently allocated in the memory heap. We then trace through the Java stacks of the currently running threads and we mark all the root objects in these stacks as "live." We must then determine all the other live objects—the ones that are reachable from the root objects.

To do this efficiently, we can perform a depth-first search (see Section 14.3.1) on the directed graph that is defined by objects referencing other objects. In this case, each object in the memory heap is viewed as a vertex in a directed graph, and the reference from one object to another is viewed as a directed edge. By performing a directed DFS from each root object, we can correctly identify and mark each live object. This process is known as the "mark" phase.

Once this process has completed, we then scan through the memory heap and reclaim any space that is being used for an object that has not been marked. At this time, we can also optionally coalesce all the allocated space in the memory heap into a single block, thereby eliminating external fragmentation for the time being. This scanning and reclamation process is known as the "sweep" phase, and when it completes, we resume running the suspended program. Thus, the mark-sweep garbage collection algorithm will reclaim unused space in time proportional to the number of live objects and their references plus the size of the memory heap.

Performing DFS In-Place

The mark-sweep algorithm correctly reclaims unused space in the memory heap, but there is an important issue we must face during the mark phase. Since we are reclaiming memory space at a time when available memory is scarce, we must take care not to use extra space during the garbage collection itself. The trouble is that the DFS algorithm, in the recursive way we have described it in Section 14.3.1, can use space proportional to the number of vertices in the graph. In the case of garbage collection, the vertices in our graph are the objects in the memory heap; hence, we probably don't have this much memory to use. So our only alternative is to find a way to perform DFS in-place rather than recursively.

The main idea for performing DFS in-place is to simulate the recursion stack using the edges of the graph (which in the case of garbage collection correspond to object references). When we traverse an edge from a visited vertex v to a new vertex w, we change the edge (v,w) stored in v's adjacency list to point back to v's parent in the DFS tree. When we return back to v (simulating the return from the "recursive" call at w), we can now switch the edge we modified to point back to w. Of course, we need to have some way of identifying which edge we need to change back. One possibility is to number the references going out of v as 1, 2, and so on, and store, in addition to the mark bit (which we are using for the "visited" tag in our DFS), a count identifier that tells us which edges we have modified.

Using a count identifier requires an extra word of storage per object. This extra word can be avoided in some implementations, however. For example, many implementations of the Java virtual machine represent an object as a composition of a reference with a type identifier (which indicates if this object is an Integer or some other type) and as a reference to the other objects or data fields for this object. Since the type reference is always supposed to be the first element of the composition in such implementations, we can use this reference to "mark" the edge we changed when leaving an object v and going to some object w. We simply swap the reference at v that refers to the type of v with the reference at v that refers to w. When we return to v, we can quickly identify the edge (v,w) we changed, because it will be the first reference in the composition for v, and the position of the reference to v's type will tell us the place where this edge belongs in v's adjacency list.